

DRIVEGUARD AI

Day 01 Summary

Environment Setup, Project Structure, PyTorch MPS & Engineering Foundations

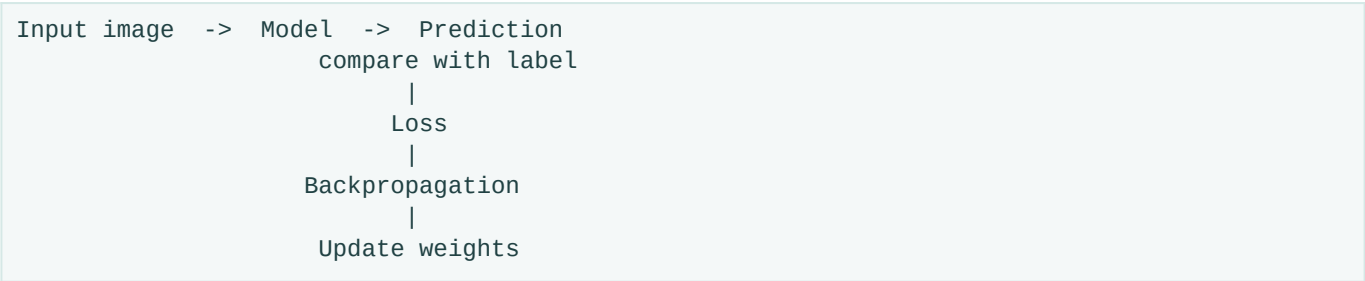
Python	PyTorch	GPU Backend	Status
3.12.14	2.13.0	MPS	DAY 1 COMPLETE

Day 1 objective

Build a clean, reproducible development environment before touching the dataset or training a model. Verify that PyTorch can use the MacBook M1 GPU and establish a project structure that can grow into a professional portfolio project.

1. Core AI Concepts

- DriveGuard AI is a supervised image-classification project: the model learns from images that already have known class labels.
- Label = the correct class (ground truth) assigned to an image.
- Prediction = the class predicted by the model. The prediction can be correct or incorrect.
- The high-level training loop is: input image -> prediction -> loss -> backpropagation -> update weights.



2. Python Environment

- Python 3.12 was installed for a modern, compatible machine-learning environment.
- A virtual environment (.venv) isolates this project's Python interpreter and packages from other projects.
- When (.venv) appears in the terminal prompt, the project environment is active.
- After activation, the python command resolves to the interpreter inside DriveGuard-AI/.venv/bin/python.

3. PyTorch & torchvision

Component	Role in DriveGuard AI
PyTorch	Deep-learning framework: tensors, neural networks, loss functions, backpropagation, optimizers, training and GPU computation.
torchvision	Computer-vision utilities for PyTorch: image transforms, dataset helpers and pretrained vision models such as MobileNet and ResNet.

4. Apple GPU / MPS Verification

- MPS (Metal Performance Shaders) is the Apple GPU backend used by PyTorch on Apple Silicon.
- MPS built = True confirms the installed PyTorch build includes MPS support.
- MPS available = True confirms the backend is available on this Mac.
- device='mps:0' confirms that the test tensor was actually moved to the Apple GPU.

```
PyTorch version: 2.13.0
MPS built: True
MPS available: True
tensor([1., 2., 3.], device='mps:0')
```

Why this matters: later, convolution and matrix operations used during neural-network training can run on the Apple GPU instead of relying only on the CPU.

5. Project Structure

```
DriveGuard-AI/
|-- .venv/          # local Python environment (ignored by Git)
|-- data/           # dataset files (ignored by Git)
|-- notebooks/      # exploration and experiments
|-- src/            # production/project Python code
|-- models/         # trained model weights (ignored by Git)
|-- results/        # metrics, plots, confusion matrices
|-- README.md       # public project documentation
|-- learning_notes.md # personal learning log
|-- requirements.txt # reproducible Python dependencies
`-- .gitignore       # files Git should not track
```

6. Git Foundations

- `git init` initializes a local Git repository.
- `git add .` stages current project changes; it does not create a commit by itself.
- `git commit` creates a local snapshot of the staged changes.
- `git push` is a separate step used later to send local commits to a remote repository such as GitHub.
- `.gitignore` keeps generated, large, environment-specific or unnecessary files out of version control.

```
git init
git add .
git status
git commit -m "Initialize DriveGuard AI project"
```

Day 1 commit: [0de2f08](#) - Initialize DriveGuard AI project

7. Important Corrections from the Day 1 Test

Label vs Prediction: A label is the known correct class; a prediction is the model's output.

python vs python3: They are command names. With `.venv` active, `python` points to the project interpreter.

PyTorch vs torchvision: PyTorch is the deep-learning framework; torchvision adds vision tools and pretrained models.

git add vs commit vs push: `add` stages changes, `commit` saves a local snapshot, `push` sends commits to a remote repository.

MPS availability vs usage: `available=True` means MPS can be used; `device=mps:0` confirms actual GPU placement.

8. Day 1 Commands Reference

```
brew install python@3.12
python3.12 -m venv .venv
source .venv/bin/activate
python --version && which python
python -m pip install --upgrade pip
pip install torch torchvision
pip install numpy pandas matplotlib scikit-learn pillow
pip freeze > requirements.txt
```

9. Day 1 Completion Checklist

[x] Python 3.12 environment created	[x] Professional project folder structure created
[x] Virtual environment activated and verified	[x] <code>.gitignore</code> configured
[x] PyTorch and computer-vision dependencies installed	[x] <code>requirements.txt</code> generated
[x] Apple MPS backend detected	[x] Initial Git commit created
[x] Tensor successfully moved to <code>mps:0</code>	[x] Core concepts reviewed through a knowledge test

NEXT: DAY 02 - DATASET EXPLORATION